Computação Móvel
Mobile Computing
CM

Tutorial 2 - Weather App

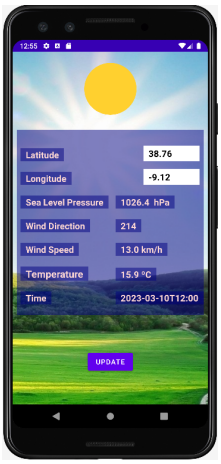
Pedro Fazenda
pedrofazenda@enautica.pt
Carlos Gonçalves

carlosgoncalves@enautica.pt

Abstract

The tutorial starts with Kotlin exercises to build programming skills. Next, students create an Android app that shows local weather and adjusts its interface for any screen size, using a REST API for data. Along the way, they'll design layouts, apply themes, support multiple languages, and improve their Kotlin skills, optionally exploring MVVM best practices.

Deadline: April 5th, 2026



Cool Weather Application

Friday, March 20th, 2026

Contents

1 Section 1 - Kotlin Exercises [AC NO, AI NO] 1

1.1 Exercise: Event Log Processing with Extension Functions and Higher-Order Functions . . . 1

1.2 Generics: A Type-Safe In-Memory Cache . . . 2

1.3 Functions and Lambdas: A Configurable Data Pipeline . . . 4

1.4 Operator Overloading: A 2D Vector Library . . . 5

2 Android - The Cool Weather App [AC NO, AI NO] 7

2.1 Laboratory Work - Weather Application . . . 7

2.2 Challenges: coordinates, XML resources and MVVM [10%] . . . 14

2.2.1 Show GPS coordinates and use XML resources . . . 14

2.2.2 Use Model-View-ViewModel (MVVM) design pattern . . . 15

3 Assisted code generation – MIP-2 [10%] 16

3.1 Objective . . . 16

3.2 Learning Goals . . . 16

3.3 Application Requirements . . . 16

3.3.1 Functional Requirements . . . 16

3.4 AI-Assisted Development with AntiGravity . . . 17

3.5 Required Project Structure . . . 17

3.6 Role of Each Markdown File . . . 18

3.6.1 README.md . . . 18

3.6.2 agents.md . . . 18

3.6.3 docs/01_overview.md . . . 18

3.6.4 docs/02_features.md . . . 18

3.6.5 docs/03_screens.md . . . 19

3.6.6 docs/04_data_model.md . . . 19

3.6.7 docs/05_navigation.md . . . 19

3.6.8 docs/06_architecture.md . . . 19

3.6.9 docs/07_api_usage.md . . . 19

3.6.10 docs/08_implementation_plan.md . . . 20

3.7 Logging AI Prompts (Recommended) . . . 21

3.8 Recommended Workflow . . . 21

3.9 Build, Deploy, and Run Using AntiGravity . . . 21

3.9.1 Required Execution Workflow . . . 21

3.9.2 Testing After Deployment . . . 22

3.10 Post-Plan Feature Extensions . . . 23

3.10.1 Extension Workflow . . . 23

3.10.2 Updating the Implementation Plan . . . 23

3.11 Evaluation Criteria . . . 24

3.12 Important Rule . . . 24

4 GIT, Commit and Push Policy 24

4.1 Minimum Expectation . . . 24

1 Section 1 - Kotlin Exercises [AC NO, AI NO]

In this section, you will find a series of Kotlin exercises designed to encourage you to read and understand the documentation before implementing a solution. As in the previous assignment, resist the temptation to use AI tools. If you cannot complete an exercise, simply move on to the next.

1.1 Exercise: Event Log Processing with Extension Functions and Higher-Order Functions

In this exercise, you will work with Kotlin's **extension functions**, **higher-order functions**, and **sealed classes**. The goal is to model a simple event log system and process events using functional programming techniques.

Instructions

1. Define a **sealed class** `Event` with the following subclasses:
   - `Login(val username: String, val timestamp: Long)` — represents a user login event.
   - `Purchase(val username: String, val amount: Double, val timestamp: Long)` — represents a purchase event.
   - `Logout(val username: String, val timestamp: Long)` — represents a user logout event.
2. Write an **extension function** on `List<Event>` called `filterByUser` that:
   - Takes a `username: String` parameter.
   - Returns only the events associated with that user.
3. Write an **extension function** on `List<Event>` called `totalSpent` that:
   - Takes a `username: String` parameter.
   - Returns the total amount spent by that user across all `Purchase` events.
   - Hint: use `filterIsInstance` and `sumOf` from the Kotlin standard library.
4. Implement a **higher-order function** `processEvents` that:
   - Takes a `List<Event>` and a lambda `handler: (Event) -> Unit`.
   - Applies the handler to each event in the list.
5. Use a `when` expression inside the lambda passed to `processEvents` to print a **human-readable description** of each event. Every branch of the `when` must be handled (exhaustiveness is enforced by the sealed class).
6. Test your implementation with the sample data below and verify the output matches the expected result.

Sample Data

```
val events = listOf(
    Event.Login("alice", 1_000),
    Event.Purchase("alice", 49.99, 1_100),
    Event.Purchase("bob", 19.99, 1_200),
    Event.Login("bob", 1_050),
    Event.Purchase("alice", 15.00, 1_300),
    Event.Logout("alice", 1_400),
    Event.Logout("bob", 1_500)
)
```

Expected Output

```
[LOGIN]    alice logged in at t=1000
[PURCHASE] alice spent $49.99 at t=1100
[PURCHASE] bob spent $19.99 at t=1200
[LOGIN]    bob logged in at t=1050
[PURCHASE] alice spent $15.0 at t=1300
[LOGOUT]   alice logged out at t=1400
[LOGOUT]   bob logged out at t=1500

Total spent by alice: $64.99
Total spent by bob: $19.99

Events for alice:
  Login(username=alice, timestamp=1000)
  Purchase(username=alice, amount=49.99, timestamp=1100)
  Purchase(username=alice, amount=15.0, timestamp=1300)
  Logout(username=alice, timestamp=1400)
```

1.2 Generics: A Type-Safe In-Memory Cache

In this exercise, you will implement a simple **generic in-memory cache** called `Cache<K, V>`, which maps keys to values of any type. This will deepen your understanding of **multi-parameter generics**, **upper bounds**, and **higher-order functions** working together with type parameters.

Instructions

1. **Create a generic class** `Cache<K : Any, V : Any>` that:
   - Internally stores entries using `MutableMap<K, V>`.
   - Has a function `put(key: K, value: V)` to insert or overwrite an entry.
   - Has a function `get(key: K): V?` to retrieve a value by key, returning `null` if the key is not present.
   - Has a function `evict(key: K)` to remove an entry from the cache.
   - Has a function `size(): Int` to return the number of entries currently stored.
2. **Add a `getOrPut` function**:
   - Signature: `getOrPut(key: K, default: () -> V): V`

- If the key already exists, return its value.
- If the key does not exist, call the `default` lambda to compute a value, store it, and return it.
- Hint: explore how `MutableMap.getOrPut` works in the Kotlin standard library before writing your own.

3. **Add a `transform` function**:
   - Signature: `transform(key: K, action: (V) -> V): Boolean`
   - If the key exists, apply `action` to its current value, update the stored value, and return `true`.
   - If the key does not exist, do nothing and return `false`.
4. **Add a `snapshot` function**:
   - Signature: `snapshot(): Map<K, V>`
   - Returns an **immutable copy** of the current cache contents.
   - The caller must not be able to modify the cache through the returned map.
5. **Test your cache with at least two different type combinations**:
   - A `Cache<String, Int>` representing word frequency counts.
   - A `Cache<Int, String>` representing a small id-to-name registry.

Expected Output

```
--- Word frequency cache ---
Size: 3
Frequency of "kotlin": 1
getOrPut "kotlin": 1
getOrPut "java": 0
Size after getOrPut: 4
Transform "kotlin" (+1): true
Transform "cobol" (+1): false
Snapshot: {kotlin=2, scala=1, haskell=1, java=0}

--- Id registry cache ---
Id 1 -> Alice
Id 2 -> Bob
After evict id 1, size: 1
Id 1 after evict -> null
```

Challenge [2%]

Add a `filterValues(predicate: (V) -> Boolean) function: Map<K, V>` that returns an immutable map containing only the entries whose values satisfy the predicate. Use it to print all words in the frequency cache with a count greater than zero.

1.3 Functions and Lambdas: A Configurable Data Pipeline

In this exercise, you will build a simple **data processing pipeline** using Kotlin **higher-order functions**, **lambdas**, and **function composition**. The goal is to process a list of raw strings through a sequence of configurable transformation stages.

Instructions

1. **Create a class** `Pipeline` that:
   - Internally stores an ordered list of transformation steps, where each step is a function of type `(List<String>) -> List<String>`.
   - Has a function `addStage(name: String, transform: (List<String>) -> List<String>)` that appends a named stage to the pipeline.
   - Has a function `execute(input: List<String>): List<String>` that runs the input through every stage in order and returns the final result.
   - Has a function `describe()` that prints the name of each stage in order.
2. **Implement a top-level function** `buildPipeline` that:
   - Accepts a lambda with `Pipeline` as its receiver.
   - Creates a `Pipeline` instance, applies the lambda to it, and returns it.
3. **Build and test a pipeline** that processes a list of raw log lines by adding the following stages in order:
   - **Trim** — remove leading and trailing whitespace from each line.
   - **Filter errors** — keep only lines that contain the word `"ERROR"`.
   - **Uppercase** — convert every remaining line to uppercase.
   - **Add index** — prefix each line with its 1-based position, e.g. `"1. ERROR: disk full"`.
4. **Print the pipeline description** using `describe()`, then run it and print the final result.

Sample Input

```kotlin
val logs = listOf(
    "  INFO: server started  ",
    "  ERROR: disk full  ",
    "  DEBUG: checking config  ",
    "  ERROR: out of memory  ",
    "  INFO: request received  ",
    "  ERROR: connection timeout  "
)
```

Expected Output

```
Pipeline stages:
  1. Trim
  2. Filter errors
  3. Uppercase
  4. Add index

Result:
  1. ERROR: DISK FULL
  2. ERROR: OUT OF MEMORY
  3. ERROR: CONNECTION TIMEOUT
```

Challenge [4%]

- Add a `compose` function to `Pipeline` that takes two existing stage names and merges them into a single new stage using Kotlin's **function composition** operator. Hint: look up `andThen` and how to compose functions of type `(T) -> T` in the Kotlin documentation.
- Add a `fork` function that runs the same input through **two independent pipelines** and returns a `Pair<List<String>, List<String»` with both results.

1.4 Operator Overloading: A 2D Vector Library

In this exercise, you will implement a **mathematical 2D vector** class called `Vec2` and use Kotlin's **operator overloading** to make it behave like a first-class numeric type. This is a classic and practical use case for operator overloading, found in game engines, physics simulations, and graphics libraries.

Instructions

1. **Define a `data class Vec2`** with two `Double` properties: `x` and `y`.
2. **Overload the following arithmetic operators** so that they work naturally between two `Vec2` instances:
   - `+` (addition): adds the components pairwise.
   - `-` (subtraction): subtracts the components pairwise.
   - `*` (scalar multiplication): accepts a `Double` on the right-hand side and scales both components.
   - Unary `-` (negation): returns a vector with both components negated.
3. **Overload the `compareTo` operator** so that two `Vec2` instances can be compared by their **magnitude** (Euclidean length). This will allow the use of `<`, `>`, `<=`, and `>=` between vectors.
   - Hint: read the documentation on how implementing `compareTo` with the `operator` keyword interacts with the `Comparable<T>` interface.
4. **Add the following non-operator member functions**:

- `magnitude(): Double` — returns the Euclidean length $\sqrt{x^2 + y^2}$.
- `dot(other: Vec2): Double` — returns the dot product $x_1x_2 + y_1y_2$.
- `normalized(): Vec2` — returns a unit vector in the same direction. Throw an `IllegalStateException` if the vector is the zero vector.

5. **Overload the `get` operator** so that `v[0]` returns `x` and `v[1]` returns `y`. Throw an `IndexOutOfBoundsException` for any other index.

6. **Test your implementation** in `main` with the sample data below.

Sample Input and Expected Output

```
fun main() {
    val a = Vec2(3.0, 4.0)
    val b = Vec2(1.0, 2.0)

    println("a          = $a")            // a          = Vec2(x=3.0, y=4.0)
    println("b          = $b")            // b          = Vec2(x=1.0, y=2.0)
    println("a + b      = ${a + b}")      // a + b      = Vec2(x=4.0, y=6.0)
    println("a - b      = ${a - b}")      // a - b      = Vec2(x=2.0, y=2.0)
    println("a * 2.0    = ${a * 2.0}")    // a * 2.0    = Vec2(x=6.0, y=8.0)
    println("-a         = ${-a}")         // -a         = Vec2(x=-3.0, y=-4.0)
    println("|a|        = ${a.magnitude()}")        // |a|        = 5.0
    println("a dot b    = ${a.dot(b)}")             // a dot b    = 11.0
    println("norm(a)    = ${a.normalized()}")
    // norm(a)    = Vec2(x=0.6, y=0.8)
    println("a[0]       = ${a[0]}")       // a[0]       = 3.0
    println("a[1]       = ${a[1]}")       // a[1]       = 4.0
    println("a > b      = ${a > b}")      // a > b      = true
    println("a < b      = ${a < b}")      // a < b      = false

    val vectors = listOf(Vec2(1.0, 0.0), Vec2(3.0, 4.0), Vec2(0.0, 2.0))
    println("Longest    = ${vectors.max()}")  // Longest    = Vec2(x=3.0,
        y=4.0)
    println("Shortest   = ${vectors.min()}")  // Shortest   = Vec2(x=1.0,
        y=0.0)
}
```

Challenge [4%]

- Overload the `times` operator a second time to support **left-hand scalar multiplication**, i.e. `2.0 * v`. Hint: this requires an **extension function** on `Double` rather than a member function on `Vec2`. Read the documentation on how operator overloading works with extension functions.

- Add a `component1()` and `component2()` operator so that `Vec2` supports **destructuring declarations**: `val (x, y) = a`.

2 Android - The Cool Weather App [AC NO, AI NO]

The **CoolWeatherApp** is designed to be a user-friendly weather application that features a customizable interface that adapts seamlessly to any size screen. Provides real-time meteorological data for a specific location, identified by its latitude and longitude coordinates. The displayed information includes **Sea Level Pressure**, **Wind Direction**, **Wind Speed**, and **Temperature**. In addition, the app shows the date and time of the recorded data, along with a visual representation of the current weather conditions, such as a sun icon indicating clear skies during daylight hours. By default, the application displays weather information for **Lisbon** (latitude: 38.076, longitude: -9.12). Users can update the location by entering new coordinates and pressing the **Update** button to retrieve and view the corresponding weather details.

This tutorial is designed to guide you through the development of the application, providing useful insights on how to proceed. Along with a few code snippets, it aims to equip students with the necessary knowledge and resources to actively fill in any missing pieces of the puzzle. As such, it is highly recommended that learners take advantage of the provided hints and proactively consult the relevant documentation to implement the missing code effectively. You can find all the required documentation at the following location:

Android Developers Guide: `https://developer.android.com/guide`

2.1 Laboratory Work - Weather Application

Obtaining Weather Information

1. **Testing the Open-Meto API**.

   In this project we will be using **Open-Meteo** keyless APIs that are free for non-commercial use. Go to the following website: `https://open-meteo.com/en/docs`, select the city of Lisbon and test the API by selecting Hourly and Daily Weather variables. You will be able to preview the variables you select on a chart and, bellow that chart, you will find the corresponding API URL that you can use to make a HTTP request and obtain a JSON response with all the information you selected. This URL will look something like this:

   ```
   https://api.open-meteo.com/v1/forecast?latitude=52.52&longitude=13.41&current_
   weather=true&hourly=temperature_2m
   ```

   Play with the different variables, read the documentation, and see how the API URL changes. Pay special attention to the information regarding WMO Weather interpretation codes (bottom of the page). We will be using these codes in our application to describe the current weather in our location (clear sky, Fog, etc.).

2. **Obtain Weather Information** Copy and Paste the API URL in your browser's address bar, execute the request, and you should receive a JSON response with the weather

information you requested in the URL string. To have a prettier view of this information, copy the JSON string and use the following JSON Pretty Print Online tool: `https://jsonformatter.org/json-pretty-print`.

Create a Project and set the Background

1. In this section, we will create a **New Project** in Android Studio to display weather information in the main activity. Go ahead and create an **Empty Views Activity**, with the name "Cool Weather APP"', in package dam_AXXXX.coolweatherapp (replace XXXX with your student number).

2. Let us use as a starting reference the Google Pixel 3 AVD. The Google Pixel 3 has a 5.5-inch OLED display with a resolution of 1080 x 2160 pixels and a pixel density of 443 pixels per inch (ppi). It has a viewport size of 393 x 786 PX with 161 actual pixel density, which means that it has 2.75 xhdpi display pixel density. Search the Web for a suitable background image with at least 393 x 786 PX. If you have to edit your images, you can use, for example, Gimp (`https://www.gimp.org/`). Resize and crop the image if the original size is greater than 393 x 786 and convert the image to png format. Copy the image to the drawable folder of the project. Image names should only contain lowercase a-z, 0-9, or underscore; otherwise, they will not appear in the Pick a Resource window. Images should be placed in the drawables directory (not any other like drawable-v24).

3. Set the image as the background for your main activity. Open the "activity_main.xml" in the res/layout folder. Open the "Split" tab in the lower left corner of the editor to switch to the Layout Editor. Select the <constraintLayout> element and in the properties panel (lower right corner) put the name of "container" in the attribute id of the element. In the "background" attribute, place the image you copied at the previous point, by pressing the rightmost vertical rectangular/rounded shape to open the Pick a Resource window and select the previous image. Run the program on the Pixel 3 AVD created in the previous tutorial.

4. Create a landscape layout and adapt the background image so that it fits perfectly to this layout. To accomplish this, crop the higher-resolution image that was used in the previous section in a way that looks good, with a size of 786 x 393.

5. Create and add a new portrait and landscape layout for different screen sizes and resolutions (for example, select a tablet). Create a new AVD and test how this layout looks on this new device. Adjust the background images appropriately.

Designing the layout

To create a layout similar to the portrait and landscape layouts depicted in Figures 1 and 2, choose the required view components from the palette. Your UI creativity can earn you an additional 10% based on its quality. Follow the steps below to build your layout:

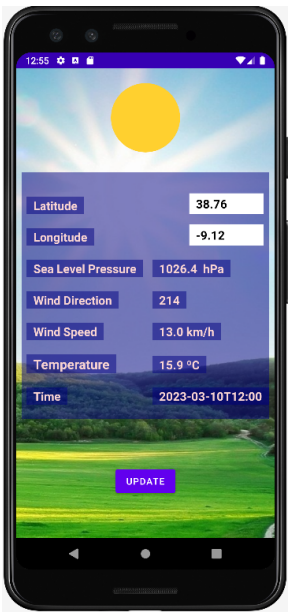


Figure 1: Portrait layout configuration for the Smart Weather App

Figure 2: Landscape layout.

1. To represent the current weather, import the color or black Vector Assets provided in the "Drawables.zip" file accompanying this tutorial. The icons in this file illustrate different weather conditions. For instance, in our layout, the sun was representing a "clear day" in Lisbon on the following date: "2023-03-10T12h00". [1]. To add the entire directory, go to "Tools/Resource Manager", press "+"(Add resources to module) and select "Import Drawables".

2. Please review the following documentation:

   `https://developer.android.com/develop/ui/views/theming/themes`

   Your application features two distinct themes: a Light Theme and a Dark Theme. Each theme should be personalized, and you should test them by adjusting the display settings found at:

   `https://developer.android.com/develop/ui/views/theming/darktheme`

   All colors must be defined in the "values/colors.xml" file. In addition, you should expand the Light Theme to include the new themes outlined in the listings 1. You can switch between themes programmatically using the code shown in the listings 2, where the "day" boolean variable is determined from the "Time" at a specific location, as well as the "Sunrise" and "Sunset" hours obtained from the weather API. We can do this later, for now just declare "day" as a **boolean** variable that you can change to test the different themes.

```
<resources xmlns:tools="http://schemas.android.com/tools">
   <!-- Base application theme. -->
   <style name="Theme.CoolWeatherApp"
          parent="Theme.MaterialComponents.DayNight.NoActionBar">
      <!-- Primary brand color. -->
```

[1] Obtained from: https://github.com/Tomorrow-IO-API/tomorrow-weather-codes

```
        <item name="colorPrimary">@color/purple_500</item>
        <item name="colorPrimaryVariant">@color/purple_700</item>
        <item name="colorOnPrimary">@color/white</item>
        <!-- Secondary brand color. -->
        <item name="colorSecondary">@color/teal_200</item>
        <item name="colorSecondaryVariant">@color/teal_700</item>
        <item name="colorOnSecondary">@color/black</item>
        <!-- Status bar color. -->
        <item name="android:statusBarColor">?attr/colorPrimaryVariant</item>
        <!-- Customize your theme here. -->
        <item name="background">@color/viewBG</item>
        <item name="android:textColor">@color/WeatherLabel</item>
        <item name="android:textSize">18sp</item>
        <item name="android:textStyle">bold</item>

    </style>

    <style name="Theme.Day" parent="Theme.CoolWeatherApp">
        <item name="android:windowBackground">@drawable/sunny_bg</item>
    </style>

    <style name="Theme.Day.Land" parent="Theme.CoolWeatherApp">
        <item name="android:windowBackground">@drawable/sunny_bg_land</item>
    </style>

    <style name="Theme.Night" parent="Theme.CoolWeatherApp">
        <item name="android:windowBackground">@drawable/night_bg</item>
    </style>

    <style name="Theme.Night.Land" parent="Theme.CoolWeatherApp">
        <item name="android:windowBackground">@drawable/night_bg_land</item>
    </style>
</resources>
```

Listing 1: Creating new Themes.

```
override fun onCreate(savedInstanceState: Bundle?) {
    // ...

    when (resources.configuration.orientation) {
        Configuration.ORIENTATION_PORTRAIT -> {
            if (day) {
                setTheme(R.style.Theme_Day)
            } else {
                setTheme(R.style.Theme_Night)
            }
        }

        Configuration.ORIENTATION_LANDSCAPE -> {
            if (day) {
                setTheme(R.style.Theme_Day_Land)
            } else {
                setTheme(R.style.Theme_Night_Land)
            }
        }
    }

    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)

    // ...
}
```

Listing 2: onCreate

3. Construct the layout using an <ImageView>, <Button>, <EditText> and <TextView> elements. Don't forget to assign a different "id" for each one. Later, we will modify the values of this element in Kotlin and for that reason we will need the id. For some elements, it is necessary to put text. Make new strings in the file "strings.xml" with the content and use them in the respective attribute. Also, define <EditText> as **android:inputType="numberDecimal|numberSigned"**.

4. Execute the program and check if the UI is close to what you expect. Small changes in the layout of the elements are expected.

5. Navigate to your "strings.xml" file and access the Strings Editor located in the top right corner. Add two languages (Portuguese and English) and assign the appropriate text for each string and language. Once this is complete, modify the language settings on your device and run the application. Verify that all strings change correctly according to the selected language.

6. Modify the boolean variable "day" and ensure that the Theme changes correctly in response.

7. Find out how to change the icon of the application launcher and change it.

Application Code

In this section, we will incorporate the business logic of our application directly into the MainActivity class. Execute the following:

1. To get started, create a new Kotlin class file named 'WeatherData.kt' and add the code shown in the listings 3. Please note that the class arrangement provided is for demonstration purposes only and should be customized to match the data retrieved from the Weather API. The **WeatherData**, **CurrentWeather** and **Hourly** classes will store the information obtained from the Weather API. The WMO enumeration **_weatherCode** contains integer codes and drawable icons representing various weather conditions. You can utilize a MAP to retrieve the name of the drawable associated with the weather code obtained from the API and use it to depict the weather in the layout of the Cool Weather Application.

```
data class WeatherData(
    var latitude: String,
    var longitude: String,
    var timezone:  String,
    var current_weather: CurrentWeather,
    var hourly: Hourly

)

data class CurrentWeather (
    var temperature : Float,
    var windspeed : Float,
    var winddirection : Int,
    var weathercode : Int,
    var time : String
)
```

```
data class Hourly (
    var time: ArrayList<String>,
    var temperature_2m: ArrayList<Float>,
    var weathercode: ArrayList<Int>,
    var pressure_msl:ArrayList<Double>
)

enum class WMO_WeatherCode(var code: Int, var image: String) {
    CLEAR_SKY(0,"clear_"),
    MAINLY_CLEAR(1, "mostly_clear_"),
    PARTLY_CLOUDY(2,"partly_cloudy_"),
    OVERCAST(3,"cloudy"),
    FOG(45,"fog"),
    DEPOSITING_RIME_FOG(48,"fog"),
    DRIZZLE_LIGHT(51, "drizzle"),
    DRIZZLE_MODERATE(53, "drizzle"),
    DRIZZLE_DENSE(55,"drizzle"),
    FREEZING_DRIZZLE_LIGHT(56,"freezing_drizzle"),
    FREEZING_DRIZZLE_DENSE(57,"freezing_drizzle"),
    RAIN_SLIGHT(61,"rain_light"),
    RAIN_MODERATE(63,"rain"),
    RAIN_HEAVY(65,"rain_heavy"),
    FREEZING_RAIN_LIGHT(66,"freezing_rain_light"),
    FREEZING_RAIN_HEAVY(67,"freezing_rain_heavy"),
    SNOW_FALL_SLIGHT(71,"snow_light"),
    SNOW_FALL_MODERATE(73,"snow"),
    SNOW_FALL_HEAVY(75,"snow_heavy"),
    SNOW_GRAINS(77, "snow"),
    RAIN_SHOWERS_SLIGHT(80, "rain_light"),
    RAIN_SHOWERS_MODERATE(81, "rain"),
    RAIN_SHOWERS_VIOLENT(82, "rain_heavy"),
    SNOW_SHOWERS_SLIGHT(85, "snow_light"),
    SNOW_SHOWERS_HEAVY(86, "snow_heavy"),
    THUNDERSTORM_SLIGHT_MODERATE(95, "tstorm"),
    THUNDERSTORM_HAIL_SLIGHT(96,"tstorm"),
    THUNDERSTORM_HAIL_HEAVY(99,"tstorm")
}

fun getWeatherCodeMap() : Map<Int,WMO_WeatherCode> {
    var weatherMap = HashMap<Int,WMO_WeatherCode>()
    WMO_WeatherCode.values().forEach {
        weatherMap.put(it.code,it)
    }
    return weatherMap
}
```

Listing 3: WeatherData.kt

2. To send an API request to **open-meto.com**, utilize the code presented in the listings 4. To use this code, you need to add the following line to your "Android-Manifest.xml" file:

```
<uses-permission android:name="android.permission.INTERNET"></uses-permission>
```

add also the following dependency to your "build.gradle" file (module: app):

```
implementation 'com.google.code.gson:gson:2.8.9
```

and also all the necessary imports to your "MainActivity.kt" file.

```
private fun WeatherAPI_Call(lat: Float, long: Float) : WeatherData {
```

```
  val reqString = buildString {
      append("https://api.open-meteo.com/v1/forecast?"
      append("latitude=${lat}&longitude=${long}&")
      append("current_weather=true&")
      append("hourly=temperature_2m,weathercode,pressure_msl,windspeed_10m")
  }
  val str = reqString.toString()
  val url = URL(reqString.toString());
  url.openStream().use {
    val request = Gson().fromJson(InputStreamReader(it,"UTF-8"),WeatherData::class.java)
    return request
  }
}
```

Listing 4: WeatherAPI_Call

3. In Android, the UI thread is responsible for drawing and updating the user interface. If any long-running task is performed on this thread, it can cause the UI to freeze, leading to an unresponsive and poor user experience. Therefore, it is recommended to run any time-consuming or blocking operations on a separate thread, usually through the use of **AsyncTask** or Thread. However, when updating the UI from a background thread is necessary, **runOnUiThread** can be used to execute code on the UI thread from the background thread. Using this method, we ensure that any UI updates occur on the UI thread and not on the background thread, thus avoiding any potential UI freezing issues. The listings 5 show how to call the weather API and update the UI.

   At the start of the application and whenever the user presses the update button, you must invoke the **fetchWeatherData** method. Updating the UI involves using the latitude and longitude coordinates to retrieve weather data and using those data to modify the weather variables, the weather image, and the background based on the local hour and sunrise/sunset information for that location.

```
private fun fetchWeatherData(lat: Float, long: Float) : Thread {
   return Thread {
     val weather = WeatherAPI_Call(lat,long)
     updateUI(weather)
   }
}


private fun updateUI(request: WeatherData) {
  runOnUiThread {

    val weatherImage : ImageView = findViewById(R.id.weatherImage)
    val pressure : TextView = findViewById(R.id.pressureValue)

    //TODO ...

    pressure.text = request.hourly.pressure_msl.get(12).toString() + "  hPa"

    //TODO...

    val mapt = getWeatherCodeMap();
    val wCode = mapt.get(request.current_weather.weathercode)
    val wImage = when(wCode) {
      WMO_WeatherCode.CLEAR_SKY,
```

```
        WMO_WeatherCode.MAINLY_CLEAR,
        WMO_WeatherCode.PARTLY_CLOUDY->if(day) wCode?.image+"day" else
            wCode?.image+"night"
        else -> wCode?.image
      }
      val res = getResources()
      weatherImage.setImageResource(R.drawable.fog)
      val resID = res.getIdentifier(wImage, "drawable",getPackageName());
      val drawable = this.getDrawable(resID);
      weatherImage.setImageDrawable(drawable);

      //TODO...

    }
}
```

Listing 5: Fetch Weather Data and Update UI

4. To test your application, modify the latitude and longitude coordinates and observe how the weather image, background, and weather parameters update accordingly. To choose suitable latitude and longitude coordinates, refer to Figures 3 and 4, which demonstrate how the altitude and longitude coordinate system operates.

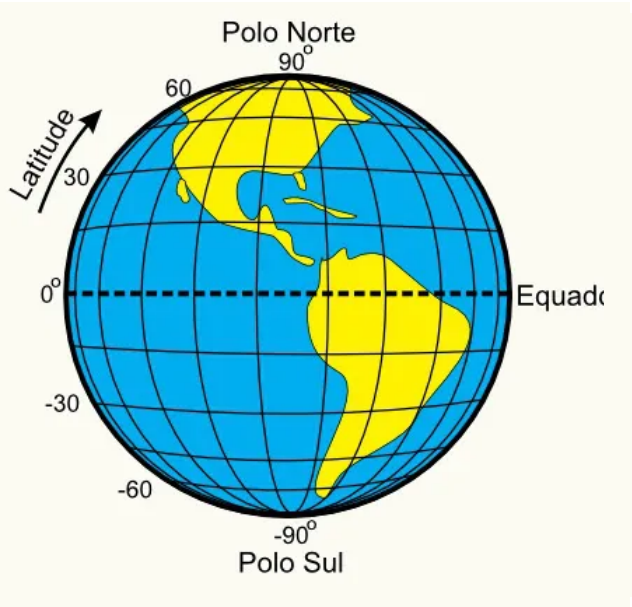


Figure 3: Latitude is a geographic coordinate that specifies a location's position north or south of the equator.

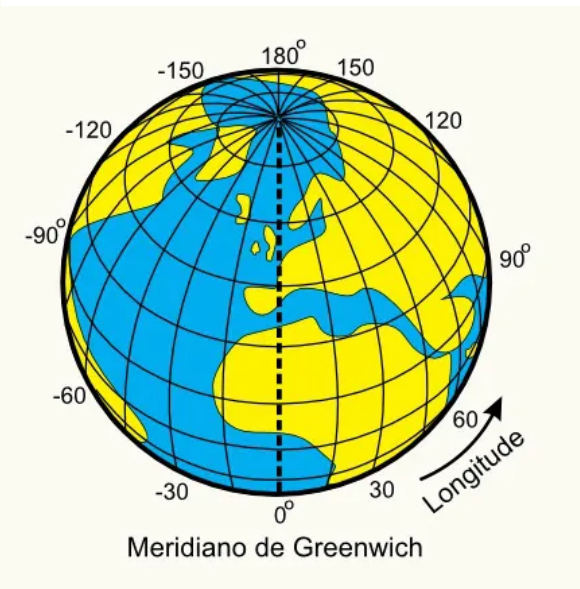


Figure 4: Longitude is a geographic coordinate that specifies a location's position east or west of the Prime Meridian.

2.2 Challenges: coordinates, XML resources and MVVM [10%]

In this section, you will find tasks that you must complete to earn up to 10% of the grade assigned to this tutorial.

2.2.1 Show GPS coordinates and use XML resources

- Upon launching the Cool Weather Application, display the latitude and longitude coordinates of the device, along with the corresponding weather information, in-

stead of using the default Lisbon coordinates.

- Rather than using a hardcoded enumeration of WeatherCodes, modify the code to retrieve the relevant information (Weather Code, Description, and Image) from a resource XML file. Refer to the slides in "CM Android - Anatomy of Android Apps.pptx" for more information on the use of array resources.

2.2.2 Use Model-View-ViewModel (MVVM) design pattern

As you dive deeper into Android development, it is important to become familiar with design patterns that can make your code more organized, scalable, and testable. One such pattern is the **MVVM (Model-View-ViewModel)** design pattern, which separates the business logic from the user interface and helps to keep your codebase organized. As part of your learning journey, you must modify the Cool Weather Application by incorporating the MVVM design pattern into it. Specifically, you will need to refactor the code in the **MainActivity** to comply with the MVVM pattern. Here are the steps you need to follow:

- Extract the business logic from MainActivity and place it in a ViewModel class.
- Use data binding to bind the View to the **ViewModel**.
- Implement **LiveData** to allow the ViewModel to communicate with the View.
- Use the Observer pattern to observe changes in the ViewModel and update the View accordingly.
- Test your application to ensure that it works as expected and that the code is clean, efficient, and organized.

By completing this task, you will gain hands-on experience in implementing the MVVM design pattern in an Android application, which will help you write better code and build more sophisticated applications in the future. Below are several helpful links that can help you complete this task.

- https://medium.com/swlh/understanding-mvvm-architecture-in-android-aa66f7e1a70b
- https://www.youtube.com/watch?v=eUQebUJLnXI
- https://www.toptal.com/android/android-apps-mvvm-with-clean-architecture
- https://developer.android.com/codelabs/basic-android-kotlin-training-repository-pattern#0
- https://www.section.io/engineering-education/implementing-a-repository-kotlin/
- https://developer.android.com/topic/libraries/view-binding
- https://developer.android.com/topic/libraries/data-binding
- https://developer.android.com/topic/libraries/architecture/lifecycle

3 Assisted code generation – MIP-2 [10%]

Your Mission Impossible Possible – MIP-2

Plan & execute with AI-Assisted Android App Development with AntiGravity.
An app generated from Markdown specifications that retrieves data from an API.

3.1 Objective

The goal of this exercise is to learn how to **design and implement an Android application using AI-assisted development** through the **AntiGravity IDE**.
Students will build a **small Android application using Kotlin and XML Views** that retrieves and displays **images from a public API**.
The assignment emphasizes a **planning-first workflow**, where the application is first specified using **Markdown documentation** before any code is generated.

3.2 Learning Goals

By completing this assignment, students will learn to:

- Use **AI-assisted development tools responsibly**
- Design software using **structured Markdown documentation**
- Follow a **planning-first development methodology**
- Build an **Android app using Kotlin and XML Views**
- Integrate with a **remote image API**
- Organize a repository using **clear architecture** and **documentation**

3.3 Application Requirements

Develop a small Android application with the following characteristics.

3.3.1 Functional Requirements

The application must:

1. Retrieve images from a **public image API** (images should have some information - it will be used later)

Possible APIs include:

- https://picsum.photos
- https://dog.ceo/dog-api
- https://thecatapi.com
- https://unsplash.com/developers

2. Display images in the interface using **XML layouts**

3. Include at least:

- One **MainActivity**
- One **RecyclerView or image grid/list**
- One **refresh action** (button or swipe refresh)

4. Be implemented in **Kotlin**

5. Use **XML Views (not Jetpack Compose)**

3.4 AI-Assisted Development with AntiGravity

You must use **AntiGravity IDE** to assist the development process.
However, the AI must be guided using **Markdown specification files** instead of ad-hoc prompts.
The development must follow this workflow:
Specification → Architecture → Implementation Plan → Code Generation
The **implementation plan acts as the bridge between design and coding**.

3.5 Required Project Structure

Your repository must follow this structure:

```
project-root
|
|- app/
|
|- docs/
|   |- 01_overview.md
|   |- 02_features.md
|   |- 03_screens.md
|   |- 04_data_model.md
|   |- 05_navigation.md
|   |- 06_architecture.md
|   |- 07_api_usage.md
|   |- 08_implementation_plan.md
|   |- prompts_log.md
|
|- agents.md
|
|- README.md
```

3.6 Role of Each Markdown File

3.6.1 README.md

High-level project description.
Include:

- Application purpose
- API used
- Screenshots
- Instructions for running the project

3.6.2 agents.md

Defines how the **AntiGravity AI agent should behave**.
Example:

```
# AI Agent Guidelines

The AI agent must follow a planning-first approach.

Rules:

- Always read the documentation inside /docs before generating code.
- Follow the architecture defined in docs/06_architecture.md.
- Generate Kotlin code only.
- UI must use XML Views.
- Do not generate large files at once.
- Generate code step-by-step following the implementation plan.
```

This file ensures the AI behaves as a **disciplined development assistant**.

3.6.3 docs/01_overview.md

High-level description of the application.
Include:

- Purpose of the application
- Target users
- General idea of how the system works

3.6.4 docs/02_features.md

List the main features.
Example:

```
Features

1. Fetch images from API
2. Display images in a list
3. Refresh images
4. Show loading indicator
```

3.6.5 docs/03_screens.md

Describe each screen.
Example:

```
Screen: Main Screen

Components:

- Toolbar
- RecyclerView with images
- Refresh button
```

3.6.6 docs/04_data_model.md

Define data structures used by the app.
Example:

```
ImageItem

id: String
url: String
title: String
```

3.6.7 docs/05_navigation.md

Describe navigation between screens.
Example:

```
MainActivity
   ↓
ImageDetailsActivity
```

3.6.8 docs/06_architecture.md

Describe the architecture used.
Example:

```
Architecture: MVVM

Layers:

UI → ViewModel → Repository → API Service
```

3.6.9 docs/07_api_usage.md

Describe the external API.
Include:

- API endpoint

- Response format
- Example JSON response

Example:

```
API endpoint

https://dog.ceo/api/breeds/image/random

Example response

{
 "message": "https://images.dog.ceo/breeds/hound-afghan/n02088094_1003.jpg",
 "status": "success"
}
```

3.6.10 docs/08_implementation_plan.md

This file contains the **step-by-step plan for implementing the application**.
The plan should break the project into **small implementation tasks**.
Example (this is an example):

```
Implementation Plan

Step 1
Create Android project with Kotlin and XML Views.

Step 2
Create data model class ImageItem.

Step 3
Implement API service to fetch images.

Step 4
Create RecyclerView adapter.

Step 5
Design activity_main.xml layout.

Step 6
Connect ViewModel to API repository.

Step 7
Display images in RecyclerView.

Step 8
Add refresh button functionality.
```

AntiGravity should follow this plan when generating code.

3.7 Logging AI Prompts (Recommended)

Students should log the **main prompts used with AntiGravity**.
Create the file: docs/prompts_log.md
Each entry should include:

```
## Prompt 1

Goal:
Generate RecyclerView adapter.

Prompt used:
<the prompt>

Result:
Short description of the generated code.
```

This helps students **reflect on how AI assisted the development process**.

3.8 Recommended Workflow

Students should follow this workflow:

1. Write the **project specifications in Markdown**
2. Define the **AI rules in agents.md and other .md files**
3. Create the **implementation plan**
4. Ask AntiGravity to **generate code step-by-step**
5. Log important prompts

3.9 Build, Deploy, and Run Using AntiGravity

All stages of **building, deploying, and running the application must be performed inside the AntiGravity IDE**.
The purpose of this requirement is to ensure that students learn how to use AntiGravity not only for code generation, but also as the **complete development environment controlling the build and execution workflow**.

3.9.1 Required Execution Workflow

Students must use AntiGravity to perform the following tasks:

1. **Project Build**

   Use AntiGravity to trigger the Gradle build process and ensure the project compiles successfully.

   The build must verify that:

   - all dependencies are resolved
   - the project compiles without errors
   - the APK is generated successfully

2. **Application Deployment**

   Deploy the application from AntiGravity to:

   - an **Android Emulator**, or
   - a **physical Android device**

   The deployment must be initiated through the IDE.

3. **Application Execution**

   Run the application using AntiGravity and verify that:

   - the application starts correctly
   - images are retrieved from the API
   - the images are displayed correctly
   - refresh functionality works
   - the application remains stable during use

3.9.2 Testing After Deployment

After deployment, students should always verify that the application's behavior is as expected, including:

- images load correctly from the API
- loading indicators appear when data is being fetched
- the RecyclerView updates correctly
- navigation (if implemented) works correctly
- favorite image logic behaves as expected

Any problems discovered during execution should trigger the following process:

```
Problem Identification → Problem Questioning → Analyse solutions →
→ Select best solution → Update Implementation Plan → Commit →
→ Fix → Commit → Test → Commit
```

where: `Problem Questioning` is asking AI why is the problem occurring and `Analyse solutions` is asking AI for solutions.

3.10 Post-Plan Feature Extensions

After completing the implementation defined in `docs/08_implementation_plan.md`, you may identify **additional improvements** or **missing functionality** that were not included in the original plan.
This is normal in real software development. New requirements often appear after the first working version of the system.
When this happens, the team must follow a **controlled extension process**.

3.10.1 Extension Workflow

When a new feature is identified:

1. **Do not immediately generate code.**
2. **Add the feature to the implementation plan.**
3. **Describe the feature clearly.**
4. **Commit the updated documentation.**
5. **Then implement the feature step-by-step with AntiGravity.**

This ensures the project remains **structured and traceable**.

3.10.2 Updating the Implementation Plan

New features must be added to: `docs/09_feature_extensions.md`, with a new section for each extension.

Each new feature should include:

- the feature name and a short description
- the implementation tasks
- the expected UI changes
- the corresponding implementation plan

New features to be implemented:

- Use of the `MVVM` pattern, if not already used
- Have a Loading Indicator (progress bar)
- Have an Image Details Screen (more information about the image)
- Support Favorite Items (FIFO queue with a maximum of 5), the user must have direct access to the five favorite items from any screen via their images
- Maintain a cache of up to 50 items (excluding favorites), keeping at least 10 items ahead and 10 behind the current position during navigation (loading Indicator must be relative to what images/items want to be loaded)
- Support Offline Access to cached items, including favorites
- Handle API errors gracefully

3.11 Evaluation Criteria

Criteria	Weight
Documentation and project structure	30%
Quality of implementation plan	20%
Android implementation	30%
Correct API integration	10%
Use of AntiGravity and prompt logging	10%

3.12 Important Rule

Even when using AI tools:
You remain responsible for understanding and validating all generated code.
AI tools should **support your learning, not replace it**.

---

4 GIT, Commit and Push Policy

Students must use, commit, and push their work **regularly** to the GitHub Classroom repositories that will be provided (or to their personal repository).
The history of the repositories must clearly reflect the **evolution of the projects**.
Large commits that include many unrelated changes should be avoided.
Instead, students should make **small, meaningful commits** that correspond to specific tasks or steps in the implementation plan.

4.1 Minimum Expectation

As a guideline, each repository should include commits related to:

- the initial project setup
- the main implementation steps defined in the implementation plan
- fixes and improvements discovered during testing
- a final commit containing final notes about the project

The commit history will be reviewed as part of the assignment evaluation.
Projects with a **single large commit or very few commits may receive a penalty**, as this does not demonstrate the development process.

Have fun and good vibes!